第三章 SQL 注入漏洞与防护——实验分析报告

任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关？

Web 应用的核心功能通常是数据的增删改查。页面输入（如表单、URL 参数）是用户向服务器传递数据的接口。服务器接收到这些输入后，会将其作为参数拼接到 SQL 语句中，发送给数据库执行。

以 **DVWA 的“用户 ID 查询”功能**为例：页面输入框中的内容直接决定了 SQL 语句中 WHERE 子句的条件。输入“1”查 ID 为 1 的用户，输入“2”查 ID 为 2 的用户。因此，页面输入直接控制了数据库查询的范围和结果。

2. 一个最小业务链路是什么？

在 SQL 注入的语境下，最小业务链路是指数据从用户端到数据库再返回的完整闭环。具体流程如下：

1. **用户输入**：用户在 DVWA 页面输入框中输入数字“1”。
2. **服务器构造查询**：后端代码（PHP）获取该输入，并拼接成 SQL 语句：SELECT * FROM users WHERE id='1';
3. **数据库执行**：数据库引擎解析并执行该 SQL 语句，在 users 表中检索数据。
4. **返回结果**：数据库将找到的用户信息（如用户名、密码哈希）返回给服务器。
5. **页面展示**：服务器将数据渲染在页面上展示给用户。

3. 用户输入是如何进入查询条件的？

用户输入是通过**字符串拼接**的方式进入查询条件的。

在后端代码中，开发人员通常直接将用户输入的变量（如$id）嵌入到 SQL 查询字符串中。例如：

```
$id = $_GET['id']; // 获取用户输入
$query = "SELECT * FROM users WHERE id='$id'"; // 将输入直接拼接到查询语句中
```

此时，用户输入的内容不再是单纯的“数据”，而是变成了 SQL 命令字符串的一部分。

4. 为什么这为 SQL 注入创造了可能性?

因为服务器端代码**无条件信任了用户输入**，将其直接拼接到 SQL 语句中，导致 **SQL 查询的结构被污染**。

正常情况下，输入应仅作为数据处理。但由于缺乏过滤，攻击者输入的特殊字符（如单引号 '）可以闭合原本的 SQL 语法结构，而后续输入的逻辑运算符（如 OR '1'='1）则会被数据库当作 SQL 代码执行。

例如:

当输入 1' OR '1'='1 时，原本的查询结构被破坏，拼接后的语句变为:
SELECT * FROM users WHERE id='1' OR '1'='1'
原本仅查询 ID 为 1 的逻辑，变成了查询“ID 为 1 **或者** 1 等于 1（恒真）”的所有记录，导致数据库泄露所有用户信息。这就是 SQL 注入的根本原因——**代码与数据未分离，导致查询结构被篡改**。

任务 2：DVWA SQL Injection Low 现象记录

1. 实验页面或实验目标

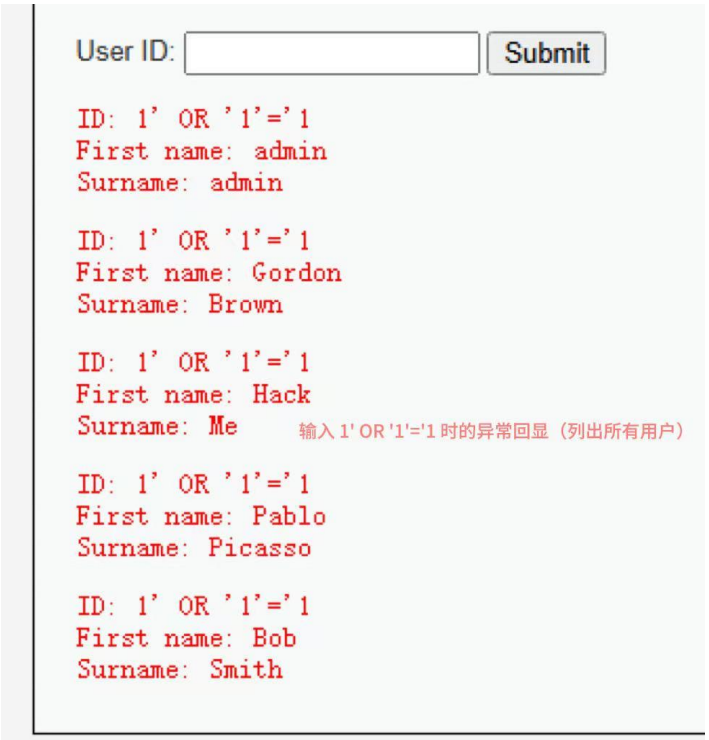
本次实验在 DVWA 平台的“SQL Injection”模块进行，安全等级设置为 **Low**。实验目标是通过输入不同的 ID 值，观察页面返回结果的差异，验证 SQL 注入漏洞的存在。

2. 普通输入与异常输入的对比

- 普通输入（图 1）：在 User ID 输入框中输入数字 **1**。
- 异常输入（图 2）：在 User ID 输入框中输入逻辑语句 **1' OR '1'='1**。

3. 返回结果的差异

- 普通输入结果：页面仅返回了 ID 为 1 的用户信息（First name: admin, Surname: admin）。
- 异常输入结果：页面返回了数据库中所有用户的列表（包括 admin, Gordon Brown, Hack Me 等）。



4. 为什么这种差异不像“正常查询变化”？

这种差异证明了**查询结构被破坏（被污染）**，而不仅仅是查询条件的变化。

- 正常逻辑：如果是正常的查询（如输入 2），结果应该仍然只有一条记录，仅仅是

ID 不同。

- 结构被污染：在异常输入中，输入的单引号'闭合了原 SQL 语句中的字符串，而 OR '1'='1'强行添加了一个恒为真的逻辑条件。
- 结论：这使得原本的查询语句 SELECT * FROM users WHERE id='...' 变成了 SELECT * FROM users WHERE id='1' OR '1'='1'。因为 '1'='1' 永远成立，数据库忽略了原本的 ID 限制，输出了全表数据。这直接支撑了“SQL 注入的根因是查询结构被污染”这一判断。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构?

SQL 注入的核心路径是“**输入→拼接→解析→执行→回显/行为差异**”：

- 用户输入（如表单参数、URL 参数）被后端代码**直接拼接到 SQL 查询语句**中（例如：SELECT * FROM users WHERE username = '$input_username' AND password = '$input_password'）；
- 拼接后的 SQL 语句被数据库解析（将输入内容误当作 SQL 语法的一部分）；
- 数据库执行解析后的查询；
- 执行结果通过页面回显（如查询结果显示在页面上）或行为差异（如页面响应时间、HTTP 状态码变化）反馈给攻击者。

2. 为什么根因是“结构被污染”而不是“字符本身危险”?

SQL 注入的**根本原因是查询结构被输入污染**，而非输入字符本身“危险”。

- 例如，输入' OR 1=1 --中的'、OR、--等字符本身是合法的 SQL 语法符号，但当它们被拼接到查询结构中时（如 SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = '...'），会改变原查询的逻辑（绕过密码验证）。
- 若输入未进入查询结构（如使用参数化查询，输入作为数据而非语法），即使包含危险字符也不会导致注入。因此，**问题在于输入被错误地拼进了查询结构**，而非字符本身。

3. Low 与 Medium 的差异在哪里？

以常见的 DVWA 靶场为例，Low 和 Medium 级别的 SQL 注入差异主要在于**输入过滤机制的强弱**：

- **Low 级别**：无输入过滤，直接拼接用户输入到 SQL 查询中（如$query = "SELECT * FROM users WHERE id = '$id'";），攻击者可直接注入（如 id=1' OR 1=1 --）。
- **Medium 级别**：对输入进行了**部分过滤**（如使用 mysql_real_escape_string 转义特殊字符，或过滤'、"等），但过滤不彻底（如未处理 OR、AND 等逻辑运算符，或未限制输入类型）。例如，Medium 级别可能将'转义为\'，但攻击者可通过**二次注入**（如先插入\'，再通过后续查询还原）或绕过转义（如使用 CHAR()函数代替字符）实现注入。
- **核心差异**：Low 无过滤，Medium 有部分过滤但未完全阻断输入进入查询结构。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里？

普通 SQL 注入（Error-Based/Union-Based）与盲注（Blind SQL Injection）的**最大差异是证据形式**：

- **普通 SQL 注入**：**有直接回显**（如查询结果直接显示在页面上，或数据库错误信息暴露查询结构）。例如，Union 注入时，攻击者可通过 UNION SELECT 将其他表的数据拼接到原查询结果中，直接看到数据（如 id=1 UNION SELECT username, password FROM users --）。
- **盲注**：**无直接回显**（页面不显示查询结果或错误信息），攻击者需通过行为差异推断数据（如页面响应时间、HTTP 状态码、页面内容变化）。例如，时间盲注通过 SLEEP(5)让数据库延迟响应（如 id=1 AND SLEEP(5) --），根据页面加载时间判断注入是否成功；布尔盲注通过 AND 1=1（页面正常）和 AND 1=2（页面异常）的差异推断数据（如 id=1 AND SUBSTRING(username,1,1)='a' --）。

5. 为什么盲注和普通注入根因相同，但证据形式不同？

盲注和普通 SQL 注入的**根因相同**（均为“输入进入查询结构导致结构污染”），但**证据形式不同**的原因是**应用程序对查询结果的输出处理方式不同**：

- 普通注入：应用程序直接回显查询结果（如将数据库查询结果输出到页面），攻击者可直接看到注入效果（如数据泄露、逻辑绕过）。

- 盲注：应用程序未回显查询结果（如页面仅显示“登录成功”或“查询无结果”，不显示具体数据），或过滤了错误信息（如关闭了数据库错误回显），攻击者只能通过间接行为（如页面响应时间、内容变化）推断注入结果。

1. SQL 注入的根因更接近 B

   A. 输入里有几个危险字符

   B. 输入进入了不该进入的查询结构

2. Blind SQL Injection 和普通 SQL 注入的最大差异更接近

   A. 根因不同 B

   B. 证据形式不同

总结：

- **输入-拼接-解析-执行-回显/行为差异链条：用户输入→后端拼接 SQL→数据库解析执行→结果回显（普通注入）或行为差异（盲注）。**
- **Low 与 Medium 差异：Low 无输入过滤，Medium 有部分过滤（如转义特殊字符）但未完全阻断注入。**
- **普通 SQL 注入与盲注差异：普通注入有直接回显（结果/错误信息），盲注无直接回显（通过行为差异推断）。**
- **根因是结构被污染而非字符本身：输入被拼进查询结构导致 SQL 逻辑改变，而非字符本身“危险”。**

任务 4：SQL 注入防护清单与解释

1. 参数化查询（预编译语句）

- **解决的问题**：SQL 注入的根本原因（用户输入与 SQL 代码未分离，导致恶意输入被当作 SQL 语句执行）。参数化查询通过将 SQL 语句的结构（如 SELECT * FROM users WHERE username = ?）与参数值（如用户输入的 admin' OR 1=1--）分开，数据库会先解析 SQL 结构，再将参数作为纯数据处理，避免恶意输入干扰 SQL 逻辑。
- **控制层次**：根因控制（直接消除 SQL 注入的产生条件）。
- **不可替代性**：无法替代输入校验（如过滤特殊字符），因为参数化查询仅解决“代码

与数据分离”问题，若输入包含非法数据（如 XSS 攻击的<script>标签），仍需输入校验拦截。

2. 输入校验

- **解决的问题**：恶意输入的暴露面（用户输入中包含 SQL 注入特征的特殊字符，如'、--、OR、SELECT 等）。通过白名单校验（如仅允许数字、字母）、黑名单过滤（如拦截'、--）或格式验证（如邮箱、手机号格式），减少恶意输入到达数据库的机会。
- **控制层次**：暴露面控制（减少攻击者可利用的输入入口）。
- **不可替代性**：无法替代参数化查询（根因控制），因为输入校验可能被绕过（如使用编码、注释符），且无法解决“代码与数据未分离”的根本问题。

3. 错误处理

- **解决的问题**：错误信息的暴露面（数据库错误信息泄露，如 SQL 语法错误、表结构信息）。例如，当 SQL 查询出错时，若直接返回数据库错误（如 MySQL syntax error near 'admin' OR 1=1--'），攻击者可通过错误信息推断数据库结构，进而构造更精准的注入语句。
- **控制层次**：暴露面控制（减少攻击者可利用的信息泄露）。
- **不可替代性**：无法替代参数化查询或输入校验，因为错误处理仅能隐藏错误信息，无法阻止 SQL 注入的发生。

4. 最小权限

- **解决的问题**：SQL 注入成功后的后果范围（攻击者利用注入漏洞执行恶意操作，如删除表、读取敏感数据）。通过限制数据库账号的权限（如仅授予查询权限，禁止 DROP、DELETE 等危险操作），即使注入成功，攻击者也无法造成严重破坏。
- **控制层次**：后果范围控制（限制攻击成功后的影响）。
- **不可替代性**：无法替代参数化查询或输入校验，因为最小权限仅能降低损失，无法阻止 SQL 注入的发生。

5. 补充措施：Web 应用防火墙（WAF）

- **解决的问题**：恶意请求的暴露面（拦截已知的 SQL 注入攻击特征，如' OR 1=1--）。WAF 通过规则匹配（如正则表达式）或行为分析（如请求频率、参数异常），在请求到达应用服务器前拦截恶意流量。
- **控制层次**：暴露面控制（减少攻击者到达应用的机会）。
- **不可替代性**：无法替代参数化查询或输入校验，因为 WAF 规则可能被绕过（如使用编码、变形攻击），且无法解决“代码与数据未分离”的根本问题。

收口项 1：SQL 注入因果链/防护关系图

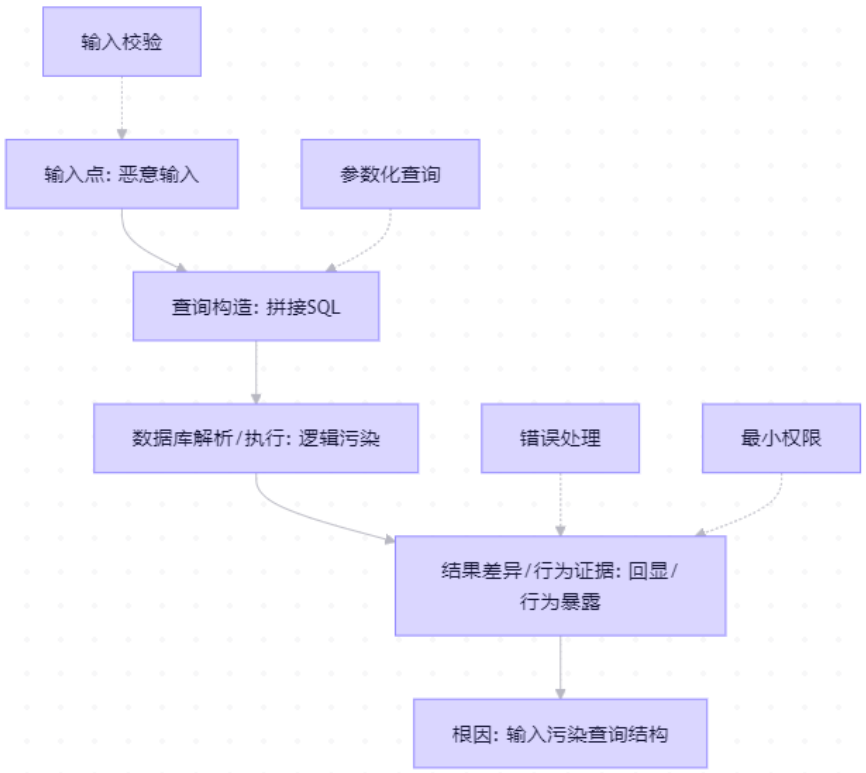
环节	描述	防护控制点	防护作用
1. 输入点	用户输入（如表单、URL 参数）包含恶意 Payload（如' OR 1=1 --）。	输入校验	过滤/拦截恶意输入（如特殊字符、SQL 关键词），减少攻击入口。
2. 查询构造	后端代码将用户输入直接拼接到 SQL 语句中（如 SELECT * FROM users WHERE username = '$input'）。	参数化查询（预编译）	分离 SQL 结构与数据，避免输入污染查询逻辑（根因控制）。
3. 数据库解析/执行	数据库将拼接后的 SQL 解析为可执行语句，恶意 Payload 改变原查询逻辑（如绕过验证）。	Web 应用防火墙	在数据库执行前，识别并拦截恶意的 SQL 流量特征。
4. 结果差异/行为证据	执行结果通过回显（如页面显示数据）或行为差异（如响应时间、HTTP 状态码）暴露漏洞。	错误处理、最小权限	- 错误处理：隐藏数据库错误信息，减少攻击者推断依据。 - 最小权限：限制数据库账号权限，降低攻击后果（如禁止 DROP 操作）。
5. 根因判断	根因是输入进入查询结构导致逻辑污染（而非字符本身危险）。		

说明：

-**因果链完整性**：覆盖“输入点→查询构造→数据库解析→结果差异→根因判断”的全流程。

-**防护对应**：每个防护点（输入校验、参数化查询、错误处理、最小权限）均对应因果链的具体环节，体现“根因控制（参数化查询）+ 暴露面/后果控制（输入校验、错误处理、最小权限）”的分层防护逻辑。

流程图：



收口项 2：学生自评表

1. **我本章最清楚的一点是什么？**
    - SQL 注入的**根因**是“代码与数据未分离”（即用户输入被当作 SQL 代码执行），以及**参数化查询**是如何通过预编译机制从根源上解决这个问题的。
2. **我最容易混淆的一点是什么？**
    - 容易混淆**输入校验**和**参数化查询**的控制层级。起初容易认为输入校验就能完全防注入，后来明白了输入校验只是“暴露面控制”（防君子不防小人），而参数化查询才是“根因控制”。
3. **我本次作业中哪一段最需要教师复核？**

- **收口项 1 中的“因果链与防护对应关系图”**。特别是关于 WAF（Web 应用防火墙）放置的位置，以及最小权限原则在因果链中的具体对应环节，希望老师确认我的逻辑归类是否准确。

4. **如果重做一次，我会优先改哪一部分？**

- 我会优先完善任务 4 的“错误处理”部分。虽然写到了要隐藏错误信息，但如果能补充具体的代码示例（如自定义错误页面 vs 数据库原生报错），会使论证更加有力。

收口项 3：AI 输出审核与人工复核记录

1. 本作业哪些内容使用过 AI 辅助

-SQL 注入因果链的逻辑梳理：在绘制“因果链/防护关系图”时，使用了 AI 辅助梳理从“输入点”到“根因判断”的标准流程，以确保没有遗漏关键环节。

-使用 AI 检查作业内容是否满足作业要求。

2. 哪些判断是你自己复核后保留的

-WAF 的位置判定：AI 最初建议将 WAF 作为主要防御手段，但我复核后认为它属于“边界防御/检测”而非根因治理，因此我将其保留在“数据库解析/执行”前的拦截环节，作为辅助手段，而非替代参数化查询。

-最小权限的归类：坚持将“最小权限”归类为“后果控制”（限制爆炸半径），而不是“预防控制”，因为即便有了最小权限，注入依然可能发生，只是危害变小了。

3. 哪些内容你发现 AI 输出不准确并进行了修正

-关于“输入校验”的有效性：AI 最初倾向于过度强调“输入校验（黑名单）”的作用。我根据课程内容修正了这一观点，明确指出黑名单容易被绕过（如宽字节注入、编码绕过），因此在我的作业中，我将其定义为“暴露面控制”而非绝对防御。

-因果链的完整性：AI 生成的初稿有时会将“结果差异”省略。我复核后认为，没有“结果差异”攻击者就无法验证漏洞，因此强制要求 AI 补充了这一环节，形成了闭环。

4. 你最终如何确认结论和证据是一一对应的

-回溯攻击链条：我通过手绘流程图（收口项 1 的图 2），将每一个防护点（如参数化查询）物理地“插入”到攻击链条的特定位置（如查询构造阶段），确保每个结论都有对应的逻辑位置。

-对照三档标准：我仔细核对了作业要求的“三档标准”，确认我的作业不仅有表（概念罗列），更有图（关系链）和具体的防护对应逻辑，从而验证了我的结论符合“满分档”的要求。

自查清单

- 我是否只在授权环境中完成实验 是
- 我的截图是否已脱敏 是
- 我是否说明了“正常查询变化”和“查询逻辑被改变”的区别 是
- 我是否把根因写成“结构被污染”，而不是只写“特殊字符” 是
- 我是否区分了普通 SQL 注入与盲注的证据差异 是
- 我是否把参数化查询、错误处理、最小权限放回不同控制层 是
- 我是否完成了关系图/关系表 是
- 我是否填写了 AI 输出审核与人工复核记录 是